

OPENSIFT EX280v4.18 Exam

Important instructions:

- utility.lab.example.com (172.25.250.14)
- master01.lab.example.com 172.25.250.12
- Server API URL: <https://api.ocp4.example.com:6443>

Questions Outline

- Configure the identity Providers for Openshift
- Configure Cluster permission
- Configure Project Permissions
- Create Groups and Configure Permissions
- Configure Quotas for Projects
- Configure and Deploy a secure route
- Scala the Application manually .
- Configure

=====

Login Instructions in exam:

```
[desktop@desktop]$ ssh opsadm@workbentch.ocp4.example.com
[opsadm@workbentch]$ cat /home/opsadm/kubeadmin-password
[opsadm@workbentch]$ oc login -u kubeadmin -p admin --server
https://api.ocp4.example.com:6443
[opsadm@workbentch]$ oc whoami
```

=====

1. Configure an identity provider

Configure your OpenShift cluster to use an **HTPasswd identity provider** with the following requirements:

- The name of the identity provider is **ex280-htpasswd**
- The name of the secret is **ex280-idp-secret**
- The user account **armstrong** is present and can log in with password **indiaco**
- The user account **collins** is present and can log in with password **verastar**
- The user account **aldrin** is present and can log in with password **moonhar**
- The user account **jobs** is present and can log in with password **eastiver**
- The user account **wozniak** is present and can log in with password **glimpse**

ANS:

- `htpasswd -c -b -B exam-user armstrong armstrong123`
- `htpasswd -b -B exam-user collins collins123`
- `htpasswd -b -B exam-user aldrin aldrin123`
- `htpasswd -b -B exam-user wozniak wozniak123`
- `htpasswd -b -B exam-user jobs jobs123`
- `oc create secret generic ex280-idp-secret --from-file htpasswd=exam-user -n openshift-config`
- `oc get oauth cluster -o yaml > oauth.yaml`
- `vim oauth.yaml`

spec:

```
identityProviders:
- name: ex280-htpasswd
  mappingMethod: claim
  type: HTTPasswd
  htpasswd:
    fileData:
      name: ex280-idp-secret
```

- `oc apply -f oauth.yaml`
- `watch oc get pods -n openshift-authentication`

→ Now login using every user with their password

=====

2. Configure cluster permission

Configure your OpenShift cluster to meet the following requirements:

- The user account **jobs** can perform cluster administration tasks
- The user account **wozniak** can create projects
- The user account **wozniak** cannot perform cluster administration tasks
- The user account **armstrong** cannot create projects
- The user account **kubeadmin** is not present.

ANS:

→ oc adm policy add-cluster-role-to-user cluster-admin jobs

→ oc edit clusterrolebindings self-provisioners

```
metadata:
  annotations:
    rbac.authorization.kubernetes.io/autoupdate: "false"

subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: User
  name: wozniak
```

→ oc login -u job -p jobs123

→ oc get nodes #you should see response

→ oc delete secrets -n kube-system kubeadmin

=====

3. Configure project permissions

Configure your OpenShift cluster to meet the following requirements:

The following projects exist:

- apollo

- manhattan
- gemini
- bluebook
- titan
- The user account **armstrong** is an administrator for project **apollo** and project **gemini**
- The user account **wozniak** can view project **titan** but not administer or delete it

ANS:

- oc new-project apollo
- oc new-project manhattan
- oc new-project gemini
- oc new-project bluebook
- oc new-project titan
- oc adm policy add-role-to-user admin armstrong -n apollo
- oc adm policy add-role-to-user admin armstrong -n gemini
- oc adm policy add-role-to-user admin armstrong -n gemini

Verify:

- Oc adm auth can-i create deployment -n apollo --as armstrong
- Oc adm auth can-i create deployment -n gemini --as armstrong
- Oc adm auth can-i view deployment -n titan --as wozniak
- Expected ans would be "yes"
- Oc adm auth can-i create deployment -n titan --as wozniak
- Expected ans would be "no"

=====

4. Configure groups

Configure your OpenShift cluster to meet the following requirements:

- The user account **armstrong** is a member of the **commander** group
- The user account **collins** is a member of the **pilot** group
- The user account **aldrin** is a member of the **pilot** group

- Members of the **commander** group have edit permission in the **apollo** project
- Members of the **pilot** group have view permission in the **apollo** project

ANS:

- oc adm groups new commander
- oc adm groups new pilot
- oc get groups.user.openshift.io
- oc adm groups add-users commander armstrong
- oc adm groups add-users pilot collins
- oc adm groups add-users pilot aldrin
- oc adm policy add-role-to-group edit commander -n apollo
- oc adm policy add-role-to-group view pilot -n apollo
- oc auth can-i create pod -n apollo --as armstrong
 - ◆ Expected ans “yes”
- oc auth can-i create pod -n apollo --as collins
- oc auth can-i create pod -n apollo --as aldrin
 - ◆ Expected ans “no”
- oc auth can-i view pod -n apollo -n collins
- oc auth can-i view pod -n apollo -n aldrin
 - ◆ Expected ans “yes”

=====

5. Configure quotas

Configure your OpenShift cluster to use quotas in the **manhattan** project with the following requirements:

- The name of the quota is **ex280-quota**
- The amount of memory consumed across all containers may not exceed **15Gi**
- The total amount of CPU consumed across all containers may not exceed **2 full cores**
- The maximum number of replication controllers does not exceed **3**
- The maximum number of pods does not exceed **3**
- The maximum number of services does not exceed **6**

ANS:

- oc project manhattan
- oc create quota ex280-quota
--hard=cpu=2,memory=15Gi,pods=3,services=6,replicationcontrollers=3
--dry-run=client -o yaml > quota.yaml

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: ex280-quota
  namespace: manhattan
spec:
  hard:
    limits.cpu: "2"
    limits.memory: "15Gi"
    pods: "3"
    replicationcontrollers: "3"
    services: "6"
```

- oc apply -f quota.yaml
- oc describe resourcequotas ex280-quota

=====

6.Configure and deploy a secure route Deploy an application called oxcart securely in the project called area51

- The application has self-signed certificate available at
"/C=US/ST=NC/L=Raleigh/O=RedHat/OU=RHT/CN=oxcart.apps.ocp4.example.com" Use
newcert command to generate required tls objects, pass the subject as argument value

- The application should be reachable at the following url
`https://oxcart.apps.ocp4.example.com`

- Application produces a valid Output

ANS:

In exam you will get newcert cmd, using which private key and public key generated. But in lab you have to manually create certificate and key annually using below command

```
→ oc new-project area51
→ oc new-app --name oxcart --image quay.io/redhattraining/hello-world-nginx
→ openssl genrsa -out oxcart.key
→ openssl req -new -key oxcart.key -out oxcart.csr
→ openssl x509 -req -in oxcart.csr -signkey oxcart.key -days 365 -out oxcart.crt
→ Oc get route # if you see route already created named oxcart, then delete it
→ oc create route edge --service oxcart --key oxcart.key --cert oxcart.crt --hostname oxcart.apps.ocp4.example.com
→ oc get route
```

=====

7. Deploy an application

Deploy the chart named **redhat-movie** in the project **aeti-service** from the repository: <http://helm.domain6.example.com/charts/>

You may use the **telnet** or **nc** commands to validate the deployment.

ANS:

```
Oc project aeti-service or oc new-project aeti-service

helm repo add custom-repo http://helm.ocp4.example.com/charts

#custom-repo is just name

helm repo list #verify repo add
```

helm install myapp custom-repo/redhat-movie #this cmd only works in exam

Oc get pods #verify app created

Oc get svc #note external ip which generated against the service

Telnet → connect → ip add → see result

IN LAB to practice perform below steps:

- helm repo add do280-repo <http://helm.ocp4.example.com/charts>
- helm search repo --versions
- Vim values.yaml #add below lines in that file

```
image:
  repository: registry.ocp4.example.com:8443/etherpad
  name: etherpad
  tag: 1.8.18
route:
  host: development-etherpad.apps.ocp4.example.com
```

- helm install example-app do280-repo/etherpad -f values.yaml
- oc get pods
- oc get route

=====

8. Configure a service account

Configure a service account in the **apples** project to meet the following requirements:

- The name of the service account is **ex280sa**
- The service account allows pods to be run as **any available use**

ANS:

- oc new-project apple or oc project apple
- oc create sa ex280sa
- oc adm policy add-scc-to-user anyuid -z ex280sa

=====

9. Deploy an application

Deploy the application called **oranges** in the **apples** project so that the following conditions are true:

- The application uses the **ex280sa** service account
- No configuration components have been added or removed
- The application produces output.

ANS:

- oc new-app --name orange --image
quay.io/redhattraining/hello-world-nginx
- oc set serviceaccount deployment/orange ex280sa
- Oc get pods

Now see the pods and verify service endpoints, is it forwarding packets to pods?
If yes then check the route and access it.

If not forwarding then check the label which is assigned to pods and selector of service, both should be the same.

- Oc get svc
- Oc get endpoints #you should not see endpoint ip add
- Oc get pods --show-labels #check the labels assigned to the orange app
- Oc edit svc <svc name> #make sure under selector, correct label has assigned
- Oc get endpoints #you should see the pod ip add

=====

10. Scale the Application manually #this will only asked in 4.14

- Scale an application called **hydra** in the project called **lerna**. The hydra application should be scaled to five times

ANS:

- oc set serviceaccount deployment/orange ex280sa
- oc new-app --name hydra --image quay.io/redhattraining/hello-world-nginx
- oc get pods
- oc get pods
- oc scale deployment hydra --replicas 5

=====

11. Configure a Secret

- Configure a secret in the **math** project and the name of the secret should be **magic**.
- The secret should have following key value pairs **ACCESS_STRING**:
ASDA142hfh-gfrhhueo-erfdk345v

ANS:

- oc new-project math
- oc create secret generic magic --from-literal
ACCESS_STRING=ASDA142hfh-gfrhhueo-erfdk345v

=====

12. Configure an application to use a secret

Configure the application called **red** in the **math** project with the following requirements:

- The application uses the secret previously created called **magic**
- The secret defines an environment variable with name **ACCESS_STRING**
- The application output no longer displays:
Sorry, application is not configured correctly

ANS:

- oc new-project math or oc project math
- oc get route
 - ◆ Access app using route add, it shows app not configured msg
- oc set env deployment/red --from secret/magic
- Oc describe pod <pod name>
- Access the application, it should not print “ **Sorry, application is not configured correctly**” message.

=====

13. Configure a network policy

Configure a network policy using the **database** and **checker** projects with the following requirements:

- The database project has a network policy with the name db-allow-mysql-conn based on pod selector label network.openshift.io/policy-group
- Connections to the database project are restricted to deployments from the checker project
- The network policy is filtered by:
 - Project selector using the team=devsecops label
 - Pod selector using the deployment=web-mysql label
- The application can establish a connection to port 3306/TCP

You can check your work by examining the logs in the **checker** project.

ANS:

- oc get pods -n database --show-labels
 - ◆ Mark the label assign to database pod
- oc get pods -n checker --show-labels
 - ◆ Mark the label assign to pod of checker namespace
- oc get ns checker --show-labels
 - ◆ Mark the label assign to namespace checker
- vim allow.yaml

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-allow-mysql-conn
  namespace: database
spec:
  podSelector:
    matchLabels:
      network.openshift.io/policy-group: database
  policyTypes:
    - Ingress
  ingress:
    - from:
      - namespaceSelector:
          matchLabels:
            team: devsecops
        podSelector:
          matchLabels:
            deployment: web-mysql
    ports:
      - port: 3306
        protocol: TCP

```

Please note: when you check the label of pod in database namespace, then you see labels, which you need to assign in your yaml file

- oc apply -f allow.yaml
- oc describe netpol -n database db-allow-mysql-conn
- Oc logs -n checker pods/<check pod name>

=====

14. Deploy an application

Deploy the application called **atlas** in the **mercury** project so that the following conditions are true:

- No configuration components have been added or removed
- The application produces output.

ANS:

- oc project mercury or oc new-project mercury
- oc get pods #you will see pod is not in running status
- oc get events -o wide | grep <pod name> #check there was events for failing the event
- oc edit deployment atlas
- Here, changed the value of memory under spec.template.contianers.resources.requests to 1Gi

=====

15. Install an operator

Install the **file-integrity** operator with the following requirements:

- The operator is installed in the **openshift-file-integrity** project
 - The approval strategy is **Automatic**
 - Cluster monitoring is enabled for the **openshift-file-integrity** project
- 16. Configure a project template**

ANS:

- oc whoami --show-console
- Open the url in firefox and login into jobs user with password
- Go to Operators › OperatorHub

Project: All Projects ▾

OperatorHub

Discover Operators from the Kubernetes community and Red Hat partners, curated by Red Hat. You can purchase commercial software through [Red Hat Marketplace](#). You can install Operators on your clusters to provide optional add-ons and shared services to your developers. After installation, the Operator capabilities will appear in the [Developer Catalog](#) providing a self-service experience.

All Items

Monitoring

Networking

OpenShift Optional

Security

Storage

Source

☐ GLS Operator Catalog
Cs (1)

Provider

☐ Red Hat (1)

All Items

1 items



GLS Operator Catalog Cs

File Integrity Operator

provided by Red Hat

An operator that manages file integrity checks on nodes.

Red Hat

OpenShift

4

admin

Project: All Projects

OperatorHub

Discover Operators from the Kubernetes community through [Red Hat Marketplace](#). You can install operators from Red Hat Marketplace or from operators provided by Red Hat. After installation, the Operator can manage the lifecycle of the application.

All Items

Monitoring

Networking

OpenShift Optional

Security

Storage

Source

☐ GLS Operator Catalog Cs (1)

Provider

☐ Red Hat (1)



File Integrity Operator

1.3.6 provided by Red Hat

Install

Channel

stable

Version

1.3.6

Capability level

☒ Basic Install

☒ Seamless Upgrades

☐ Full Lifecycle

☐ Deep Insights

☐ Auto Pilot

Source

GLS Operator Catalog Cs

[OperatorHub](#) > Operator Installation

Install Operator

Install your Operator by subscribing to one of the update channels to keep the Operator up to date. The strategy determines either manual or automatic updates.

Update channel * ⓘ

stable ▾

Version *

1.3.6 ▾

Installation mode *

- ☒ All namespaces on the cluster (default)
Operator will be available in all Namespaces.
- ☐ A specific namespace on the cluster
Operator will be available in a single Namespace only.

Installed Namespace *

- ☒ Operator recommended Namespace:  openshift-file-integrity



File Integrity Operator

provided by Red Hat

Provided APIs

 File Integrity

FileIntegrity is the Schema for the fileintegrity API

 FileIntegrityNodeStatus

FileIntegrityNodeStatus defines the status of a specific node

→ Click Install to install the operator



 4 admin ▾



File Integrity Operator

file-integrity-operator.v1.3.6 provided by Red Hat



Installed operator: ready for use

[View Operator](#)

[View installed Operators in Namespace openshift-file-integrity](#)

→ Please follow Guided Exercise “Install Operators by Using the Web Console” ch:7

=====

16. Configure Template

Configure your OpenShift cluster so that new projects are created with limits using the following requirements:

- The name of the limit range is **PROJECT_NAME-limits**
where **PROJECT_NAME** is the name of the project created using **oc new-project**
- The amount of memory consumed by a single container is:
 - Minimum: **128Mi**
 - Maximum: **1Gi**
 - Default: **512Mi**
 - Default request: **256Mi**

ANS:

```
oc adm create-bootstrap-project-template -oyaml > project-template.yaml
```

```
Vim project-template.yaml
```

Add below options right above parameters section

```

- apiVersion: v1
  kind: LimitRange
  metadata:
    name: ${PROJECT_NAME}-limits
  spec:
    limits:
      - type: Container
        min:
          memory: "128Mi"
        max:
          memory: "1Gi"
        default:
          memory: "512Mi"
        defaultRequest:
          memory: "256Mi"
    parameters:
      - name: PROJECT_NAME
      - name: PROJECT_DISPLAYNAME
      - name: PROJECT_DESCRIPTION
      - name: PROJECT_ADMIN_USER
      - name: PROJECT_REQUESTING_USER

```

oc apply -f project-template.yaml -n openshift-config

oc get template -n openshift-config

```

[student@workstation ~]$ oc get template -n openshift-config
NAME                                DESCRIPTION    PARAMETERS    OBJECTS
project-request                     5 (5 blank)   3

```

oc edit projects.config.openshift.io cluster

```

spec:
  projectRequestTemplate:
    name: project-request

```

Watch oc get pods -n openshift-apiserver

Every 2.0s: oc get pods -n openshift-apiserver

NAME	READY	STATUS	RESTARTS	AGE
apiserver-5957cb7f6d-9g566	0/2	Pending	0	5s
apiserver-6446658fb6-26747	2/2	Terminating	20	303d

17. Create a cron job

Create a cron job with the following requirements:

- The cron job name is **ipsum**
- The cron job uses the image:
registry.domain6.example.com/library/job-runner:latest
- The cron job runs every minute
- The cron job successful job history limit is **10**
- The cron job runs in a project called **lorem**

ANS:

- Oc project lorem
- Oc create coronjob --image
registry.domain6.example.com/library/job-runner:latest --schedule "* * * * *
- Oc get cronjob
- Oc edit cronjob
- Under spec.successfulJobHistoryLimit change value to 10